HybridSynchAADL Manual

1 Introduction

The HybridSynchAADL tool is a formal modeling and anlaysis tool for virtually synchronous cyber-physical systems (CPSs) with complex control programs, continous behaviors, bounded clock skews, network delays, and execution times. The tool provides the HybridSynchAADL language for modeling virtually synchronous CPSs using the avionics modeling standard AADL [28], and a property specification language for specifying bounded reachability and invariant properties of HybridSynchAADL models. The tool is a plugin of OSATE, (`https://osate.org/`) the standard tool environment for AADL. Our tool provides various formal analysis methods, includign symbolic reachability analysis and randomized simulation, using Maude combined with SMT solving.

The architecture of the tool is illustrated in Figure 1. The tool first statically checks whether a given model is a valid model that satisfies the syntactic constraints of HybridSynchAADL. It uses OSATE's code generation facilities to synthesize the corresponding Maude model from the HybridSynchAADL model. Finally, our tool invokes Maude and an SMT solver to check whether the model satisfies given invariant and reachability requirements with respect to the formal semantics of HybridSynchAADL. The `Result` view in OSATE displays the results of the analysis in a readable format. The tool is available at `https://hybridsynchaadl.github.io/download`.

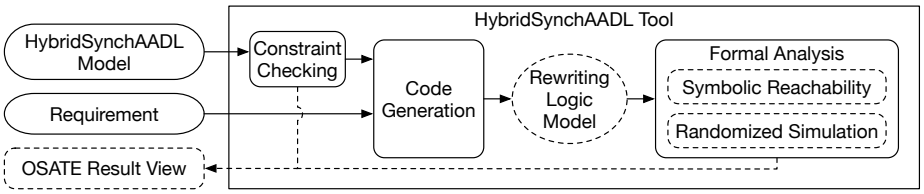

Fig. 1. The architecture of the HybridSynchAADL tool.

2 HybridSynchAADL Language

2.1 AADL

The *Architecture Analysis & Design Language* (AADL) [28] is an industrial modeling standard used in avionics, aerospace, automotive, medical devices, and robotics to describe an embedded real-time system. In AADL, a component *type*

specifies the component's *interface* (e.g., ports) and *properties* (e.g., periods), and a component *implementation* specifies its internal structure as a set of *subcomponents* and a set of *connections* linking their ports. An AADL construct may have *properties* describing its parameters, declared in *property sets*. The OSATE modeling environment provides a set of Eclipse plug-ins for AADL. We refer to the textbook [28] for more information on AADL.

An AADL model describes a system of hardware and software components. We focus on the software components, since we use AADL to specify *synchronous designs*. Hardware components include: *processor* components that schedule and execute threads, *memory* components, *device* components, and *bus* components that interconnect processors, memory, and devices. Software components include *threads* that model the application software to be executed; *process* components defining protected memory that can be accessed by its thread and data subcomponents; and *data* components representing data types. *System* components are the top-level components. A port is either a *data* port, an *event* port, or an *event data* port. Event ports and event data ports support queuing of, respectively, "events" and message data, while *data* ports only keep the latest data. *Modes* represent the operational states of components. A component can have mode-specific property values, subcomponents, etc. Mode transitions are triggered by events.

Thread behavior is modeled as a guarded transition system with local variables using AADL's *Behavior Annex* [29]. The actions performed when a transition is applied may update local variables, call methods, and/or generate new outputs. Actions are built from basic actions using sequencing, conditionals, and finite loops. When a thread is activated, transitions are applied; if the resulting state is not a *complete* state, another transition is applied, until a *complete* state is reached. The *dispatch protocol* of a thread determines when a thread is executed. In particular, a *periodic* thread is activated at fixed time intervals. See Figures 15–14 below for some examples of AADL models.

2.2 HybridSynchAADL Modeling Language

The HybridSynchAADL language models virtually synchronous cyber-physical systems in AADL. The language can specify environments with continuous dynamics, synchronous designs of distributed controllers, and nontrivial interactions between controllers and environments with respect to imprecise local clocks and sampling and actuation times.

The HybridSynchAADL language is a subset of AADL extended with the following property set `Hybrid_SynchAADL`. We use a subset of AADL without changing the meaning of AADL constructs or adding new a annex—the subset has the same meaning for synchronous models and asynchronous distributed implementations—so that AADL experts can easily develop and understand HybridSynchAADL models.

```
property set Hybrid_SynchAADL is
  Synchronous: inherit aadlboolean applies to (system, process, thread);
```

```
  isEnvironment: inherit aadlboolean applies to (system);
  ContinuousDynamics: aadlstring applies to (system);
  Max_Clock_Deviation: inherit Time applies to (system, thread);
  Sampling_Time: inherit Time_Range applies to (system, thread);
  Response_Time: inherit Time_Range applies to (system, thread);
end Hybrid_SynchAADL;
```

Top-level System Component. There are two kinds of components in HybridSynchAADL. *Environment* are system components with continuous dynamics specified using continuous functions or ordinary differential equations. *Discrete Controllers* are usual AADL software components in the synchronous AADL subset [10, 13] of AADL. The top-level system component declares the following properties to state that the model is a synchronous design and to declare the period of the system, respectively.

```
Hybrid_SynchAADL::Synchronous => true;
Period => period;
```

Environment Components. Environment components specify physical environments of Hybrid PALS models [39]. An environment component involves real-valued variables that continuously change over time according to their dynamics. An environment component can have different *modes* to specify different continuous behaviors. A command from a discrete controller may change the mode of the environment or the value of a variable. HybridSynchAADL uses existing AADL constructs, extended with the property set `Hybrid_SynchAADL`, to model such environments. Each environment component declares the following component property to state that it specifies a physical environment.

```
Hybrid_SynchAADL::isEnvironment => true;
```

Continuous Dynamics. Data subcomponents are used to specify the state of an environment component. An environment component contains data subcomponents of type `Base_Types::Float`.[1] The values of these data subcomponents change continuously over time. The continuous dynamics of these values are specified using either ordinary differential equations (ODEs) or continuous real functions, using the component property:

```
Hybrid_SynchAADL::ContinuousDynamics => "continuous_dynamics";
```

Consider a set of ODEs over n variables $x_1, \ldots, x_n$, say, $\dot{x_i} = e_i(x_1, \ldots, x_n)$, where e_i denotes an expression over the variables $x_1, x_2, \ldots, x_n$, for $1 \leq i \leq n$. In HybridSynchAADL, they are written as a semicolon-separated string of the following form, where `d/dt(`x`)` denotes the derivative $\dot{x}$:

[1] Since an environment models a physical object, its state involves real numbers. We use `Base_Types::Float` to denote real numbers, instead of defining a new data type.

```
d/dt(x_1) = e_1(x_1,...,x_n); ... ; d/dt(x_n) = e_n(x_1,...,x_n);
```

If a closed-form solution of ODEs is already known, we can directly specify concrete continuous functions, which are parameterized by a time parameter t and the initial values $x_1(0), \ldots, x_n(0)$ of the variables $x_1, \ldots, x_n$. They are written as a semicolon-separated string of the following form:[2]

```
x_1(t) = e_1(t,x_1(0),...,x_n(0)); ... ; x_n(t) = e_n(t,x_1(0),...,x_n(0));
```

Sometimes an environment component may include real-valued parameters or state variables that have the same constant values in each iteration, and can only be changed by a controller command; their dynamics can be specified as `d/dt(x) = 0` or `x(t) = x(0)`, and can be omitted in HybridSynchAADL.

Mode Transitions. In Hybrid PALS, an environment component can have different continuous behaviors. In HybridSynchAADL, this can be specified by assigning different values to `Hybrid_SynchAADL::ContinuousDynamics` in different modes, using the following AADL syntax:

```
Hybrid_SynchAADL::ContinuousDynamics =>
      "continuous_dynamics_1" in modes (mode_1),
      ...
      "continuous_dynamics_n" in modes (mode_n);
```

Modes and mode transitions are declared in the usual AADL way: a command from a controller through an input port can trigger a mode transition, and change the continuous dynamics of the environment accordingly.

Sampling and Updating Data. An environment component interacts with discrete controllers by sending its state values, and by receiving actuator commands that may update the values of state variables. This behavior is specified in HybridSynchAADL using *connections between ports and data subcomponents.*

- A connection from a data subcomponent inside the environment to an output data port declares that the value of the data subcomponent is "sampled" by a discrete controller through the output port of the environment component.
- A connection from an environment's data input port or event data input port to a data subcomponent inside the environment declares that a controller command arrived at the input port updates the value of the data subcomponent of the environment component.

When a discrete controller sends actuator commands, some input ports of the environment component may receive no value (more precisely, some "don't care" value $\perp$). In this case, the behavior of the environment is unchanged.

[2] We may use any name for the time parameters ('`t`' above).

Discrete Components. Like in Synchronous AADL, a (discrete) controller is specified using the behavioral and structural subset of AADL, namely, hierarchical system, process, and thread components; data subcomponents; ports and connections; and thread behaviors defined using the AADL *behavior annex* [29]. The hardware, memory, and scheduling features of AADL is not considered in HybridSynchAADL, because they are not relevant to synchronous *designs*.

Dispatch. The execution of an AADL thread is specified by the *dispatch protocol*. A thread with an *event-trigerred* dispatch (such as aperiodic, sporadic, timed, or hybrid dispatch protocols) is dispatched when it receives an event. Since all "controller" components are executed in lock-step in HybridSynchAADL, each thread must have *periodic* dispatch by which the thread is dispatched at the beginning of each period. The periods of all the threads are identical to the period declared in the top-level component. In AADL, this behavior is declared by the thread component property:

```
Dispatch_Protocol => Periodic;
```

Timing Properties. In HybridSynchAADL, a controller component interacts with its environment according to its sampling and actuating times. A controller receives (samples) the state of the environment at some sampling time, and sends a controller command to (actuates) the environment at some actuation time. Sampling and actuating times are defined with respect to the imprecise local clock of the controller that may differ from the "ideal clock" by up to the maximal clock skew. These time values are declared by the component properties:[3]

```
Hybrid_SynchAADL::Max_Clock_Deviation => time;
Hybrid_SynchAADL::Sampling_Time => lower bound .. upper bound;
Hybrid_SynchAADL::Response_Time => lower bound .. upper bound;
```

The upper sampling time bound must be strictly smaller than the upper bound of actuation time, and the lower bound of actuation time must be strictly greater than the lower bound of sampling time. Also, the upper bounds of both sampling and actuating times must be strictly smaller than the maximal execution time to meet the (Hybrid) PALS constraints [39].

Initial Values and Parameters. In AADL, *data* subcomponents represent data values, such as Booleans, integers, and floating-point numbers. The initial values of data subcomponents and output ports are specified using the property:

```
Data_Model::Initial_Value => ("value");
```

Sometimes initial values can be *parameters*, instead of concrete values. E.g., you can check whether a certain property holds from initial values satisfying a certain constraint for those parameters (see Section 4). In HybridSynchAADL, such unknown parameters can be declared using the following AADL property:

[3] For sampling and actuating times, we consider time intervals, instead of exact values, because measuring times may involve some numerical errors.

```
Data_Model::Initial_Value => ("param");
```

Communication. There are three kinds of ports in AADL: *data* ports, *event* ports, and *event data* ports. Event and event data ports can trigger the execution of threads, whereas data ports cannot. In HybridSynchAADL, controller and environment components may have all three kinds, but the connections are constrained for synchronous behaviors: no connection is allowed between environments, or between environment components and the enclosing system components.

Connections Between Discrete Controllers. All (non-actuator) outputs of discrete components generated in one iteration are available to the receiving discrete components at the beginning of the *next* iteration. As explained in [9, 12], *delayed connections between data ports* meet this requirement. Therefore, two discrete components can be connected only through data ports with delayed connections, declared by the connection property:

```
Timing => Delayed
```

Connections Between Controller and Environment. In HybridSynchAADL, interactions between a controller and an environment component occur *instantaneously* at the times specified by the sampling and actuating times of the controller.[4] Because an environment does not "actively" send data for sampling, every output port of an environment must be a data output port.

On the other hand, any types of input ports, such as data, event, event data ports, are available for environment components. Specifically, a discrete controller can trigger a mode transition of an environment through event ports. Therefore, no extra requirement is needed for connections, besides the usual constraints for port to port connections in AADL.

2.3 Property Specification Language

HybridSynchAADL's *property specification language* allows the user to easily specify invariant and reachability properties in an intuitive way, without having to understand Maude or the formal representation of the models. Such properties are given by propositional logic formulas whose atomic propositions are AADL Boolean expressions. Because HybridSynchAADL models are infinite-state systems, we only consider properties over behaviors up to a given time bound.

Atomic propositions are given by AADL boolean expressions in the AADL Behavior Annex syntax. Each identifier is fully qualified with its component path in the AADL syntax. A "named" atomic proposition can be declared using AADL Boolean expressions with an identifier as follows:

[4] More precisely, processing times and delays between environments and controllers are modeled using sampling and actuating times.

```
proposition [id]: AADL Boolean Expression
```

Such user-defined propositions can appear in propositional logic formulas, with the prefix ? for parsing purposes, for invariant and reachability properties.

We can simplify component paths that appear repeatedly in conditions using component scopes. A *scoped expression* of the form

$$path \mid exp$$

denotes that the component path of each identifier in the expression *exp* begins with *path*. For example, c_1 . c_2 | ((x_1 > x_2) and (b_1 = b_2)) is equivalent to (c_1 . c_2 . x_1 > c_1 . c_2 . x_2) and (c_1 . c_2 . b_1 = c_1 . c_2 . b_2). These scopes can be nested so that one scope may include another scope. For example, c_1 | ((c_2 | (x > c_3 . y)) = (c_4 | (c_5 | b))) is equivalent to the expression (c_1 . c_2 . x > c_1 . c_2 . c_3 . y) = c_1 . c_4 . c_5 . b.

An invariant property is composed of an identifier *name*, an initial condition φ_{init}, an invariant condition φ_{inv}, and a time bound τ_{bound}, where φ_{init} and φ_{inv} are in propositional logic. Intuitively, the invariant property holds if for every (initial) state satisfying the initial condition φ_{init}, all states reachable within the time bound τ_{bound} satisfy the invariant condition φ_{inv}.

```
invariant [name]: φ_init ==> φ_inv in time τ_bound
```

A *reachability property* (the dual of an invariant) holds if a state satisfying φ_{goal} is reachable from some state satisfying the initial condition φ_{init} within the time bound τ_{bound}. It is worth noting that a reachability property can be written as an invariant property by negating the goal condition.

```
reachability [name]: φ_init ==> φ_goal in time τ_bound
```

3 HybridSynchAADL Tool's Functionality

This section introduces the HybridSynchAADL tool supporting the modeling and formal analysis of HybridSynchAADL models. The tool is an OSATE plugin which: (i) provides an intuitive language to specify properties of models, (ii) synthesizes a rewriting logic model from a HybridSynchAADL model, and (iii) performs various formal analyses using Maude combined with SMT solving.

3.1 Maude Preferences and PSPC Wizard

The HybridSynchAADL tool requires Maude the underlying formal analysis tool. To use our tool, the path for Maude needs to be set. Go to `Windows` ⇒ `Preferences` in the top menu, and choose `Maude Preferences` in the left-hand side of the `Preferences` window. Write the Maude directory and the Maude executable, accordingly as illustrated in Figure 2.

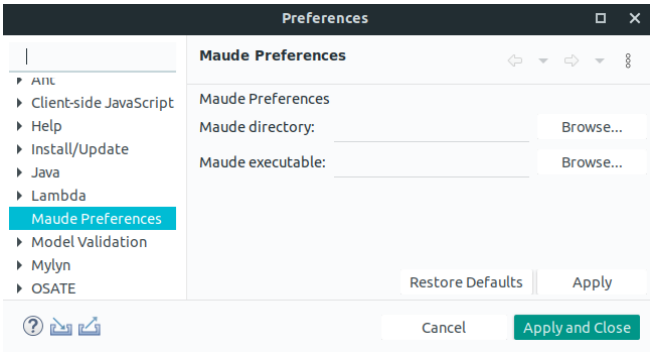

Fig. 2. Interface of the `Maude Prefernces` page

To analyze model, the user need to get the instance file. OSATE provides a facility to generate an instance file from an AADL model file. Open `Windows` ⇒ `Show View` in the top menu, and choose `Outline`. In the `Outline` view, right click on the system implementation and click `Instantiate`. The generated instance file is located in "packages/instances".

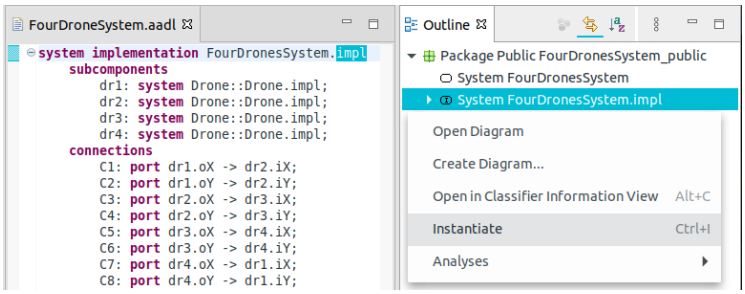

Fig. 3. The `Outline` view.

The tool provides a simple way to create a property specification (PSPC) language file. Open `New` ⇒ `Others` in the top menu, and choose `HybridSynchAADL` ⇒ `HybridSynchAADL Property Specification Language`. As shown in Figure 4, you can choose any AADL instance from the wizard. The generated PSPC file is located in the "requirement" directory. The PSPC file contains the name of the model and path of the instance file as shown in Figure 5.

3.2 Tool Interface

Figure 6 shows the interface of our tool. The left editor shows the AADL code of `FourDronesSystem` in Section 4.2, the bottom right editor shows its graphical rep-

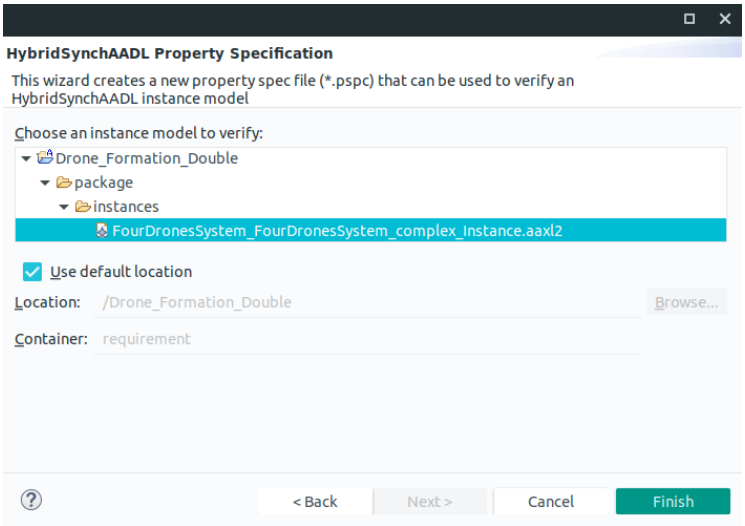

Fig. 4. The PSPC wizard.

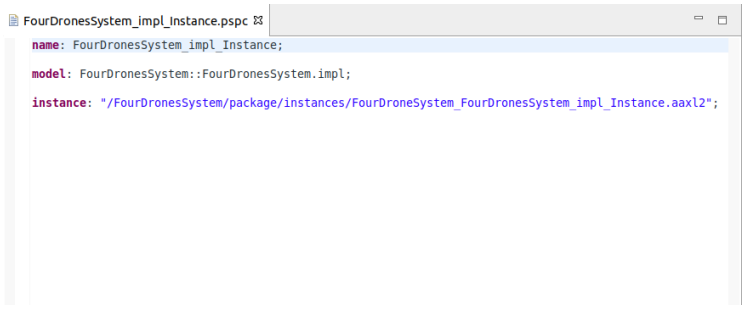

Fig. 5. The created PSPC file.

resentation, and the top right editor shows two properties in the property specification language. The HybridSynchAADL menu contains three items for constraint checking, code generation, and formal analysis. The `Portfolio Analysis` item has been already clicked, and the HybridSynchAADL Result view at the bottom displays the analysis results in a readable format.

3.3 Constraints Checking and Code Generation

By syntactically validating a HybridSynchAADL model, we ensure that the model satisfies all the syntactic constraints of HybridSynchAADL, and thus the corresponding Maude model is executable. For example, environment components (with `Hybrid_SynchAADL::isEnvironment`) can only contain data subcomponents of type `Base_Types::Float`, and must declare the continuous dynamics

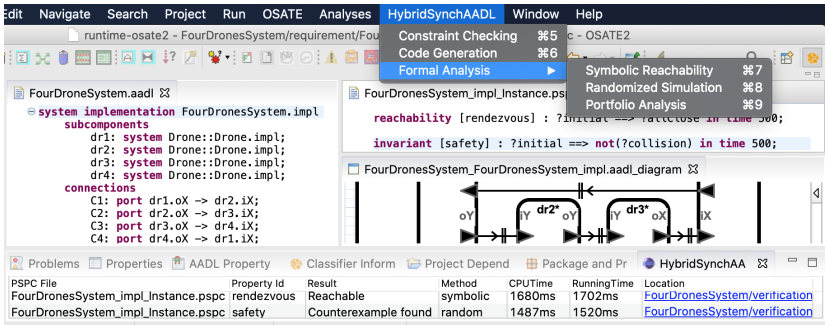

Fig. 6. Interface of the HybridSynchAADL tool.

using `Hybrid_SynchAADL::ContinuousDynamics`. The tool checks other "trivial" constraints that are assumed in the semantics of HybridSynchAADL; e.g., all connections between discrete components should be delayed. When the model satisfies all the syntactic constraints, the tools shows the pop-up window as illustrated in Figure 7. When there are the connection which is not delayed the tool shows an error message in the `Problem` view as illustrated in Figure 8.

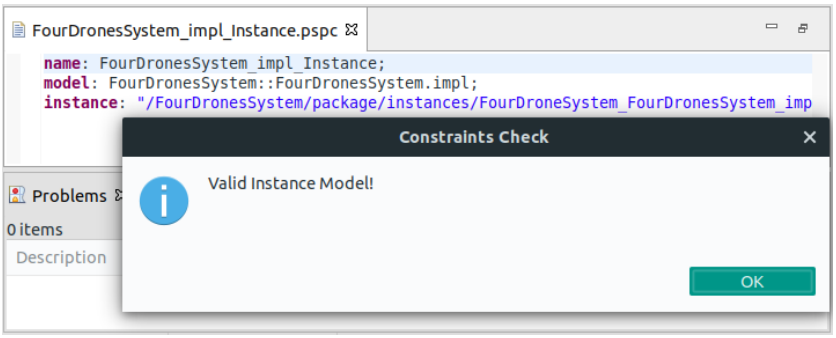

Fig. 7. Constraints Checking: The valid HybridSynchAADL model.

By synthesizing a rewriting logic model from a HybridSynchAADL model, the tool generate Maude code in `verification` directory. Like the syntactic constraints of HybridSynchAADL, there exist the constraints when generating Maude code. For example, all features does not support `access`. When the HybridSynchAADL model contains data access feature, the tool shows an error message as illustrated in Figure 10. When the model is valid, the tool shows the pop-up window like Figure 9.

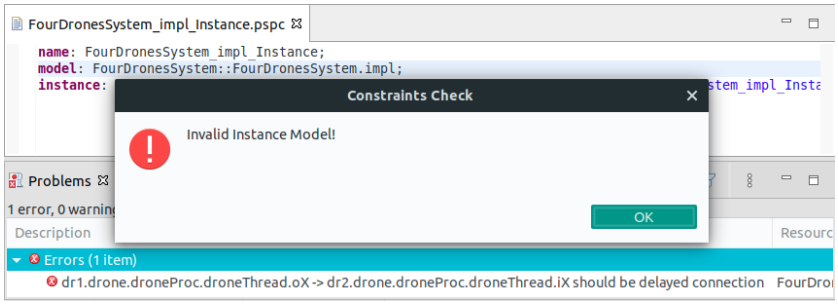

Fig. 8. Constraints Checking: The invalid HybridSynchAADL model.

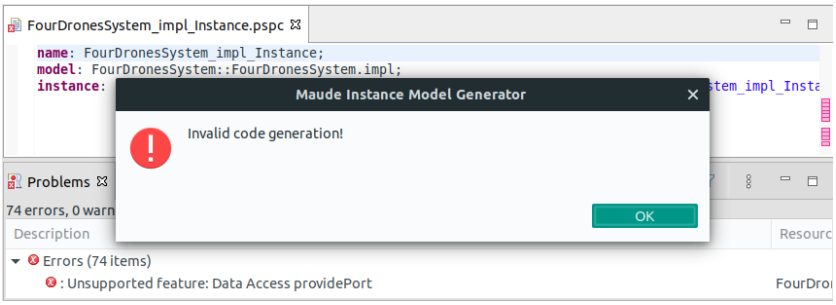

Fig. 9. Code Generation: The invalid HybridSynchAADL model.

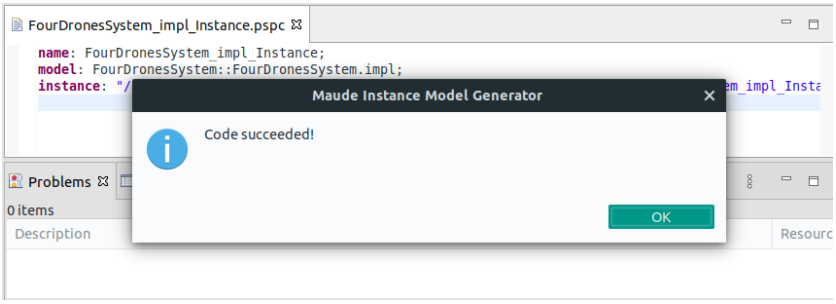

Fig. 10. Code Generation: The valid HybridSynchAADL model.

3.4 Running Formal Analysis.

Symbolic reachability analysis can verify that all possible behaviors—imposed by sensing and actuation times based on imprecise clocks—satisfy a given requirement;[5] if not, a counterexample is generated. *Randomized simulation* repeatedly executes the model (using Maude) until a counterexample is found, by randomly

[5] Symbolic analysis only supports (nonlinear) polynomial continuous dynamics, since the underlying SMT solver, Yices2, does not support general classes of ODEs.

choosing concrete sampling and actuating times, initial values of the state variables, nondeterministic transitions, etc.

Our tool also provides *portfolio analysis* that combines symbolic reachability analysis and randomized simulation. HybridSynchAADL runs both methods in parallel using multithreading, and displays the result of the analysis that terminates first. Symbolic reachability analysis can guarantee the absence of a counterexample, whereas randomized simulation is effective for finding "obvious" bugs. Portfolio analysis combines the advantages of both approaches.

When one of the formal analysis methods are chosen, the user can set the corresponding parameters in the `HybridSynchAADL Analysis` configuration in the `Run Configurations` window.

- `Random seed`: The initial random seed for random simulation.
- `Default minimum bound of "param"`: a minimum value of parameterized data subcomponents to be generated by randomized simulation.
- `Default maximum bound of "param"`: a maximum value of parameterized data subcomponents to be genereated by randomized simulation.
- `Loop bound`: The maximum bound of loop unrolling for symbolic reachability analysis.
- `Transition bound`: The maximum number of transition changes in one round for symbolic reachability analysis.
- `PropSpec File` indicates the path of the target PSPC file.
- `Timeout` indicates timeout value in seconds, where 'infinity' denotes no timeout.

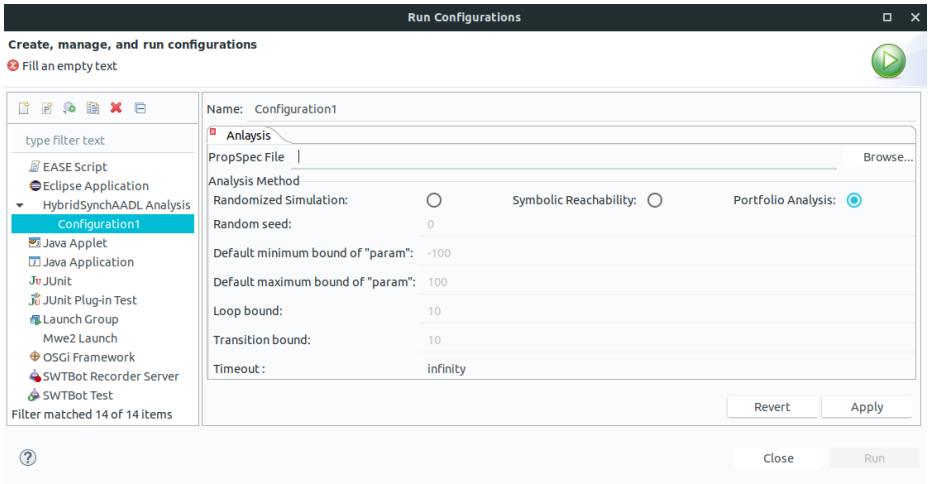

Fig. 11. A HybridSynchAADL analysis configuration.

The tool shows the results of the analysis in the HybridSynchAADL Result view as illustrated in Figure 12. The `Method` column shows which formal

analysis method is used to obtain the result. For portfolio analysis, it shows one terminated first. A file in the `Location` column contains counterexamples or witness found by the formal analysis, if any. They are given as a sequence of synchronous steps. E.g., for the counterexample of safety in Figure 12, the component *dr3* has two subcomponents *ctrl.ctrlProc.cThread* and *env*, and the value of data subcomponent *velx* is -5126.

FourDronesSystem_impl_Instance-random-safety.txt

```
(
Time: 0
FourDronesSystemimplInstance ->[
  dr1 ->[
    (ctrl . ctrlProc . cThread) ->[
      variables: none
      currState: init]
    env ->[
      variables: (velx |=> -5.1264425029806889e+3),(vely |=>
    5.6822869870071963e+3),(x |=> 1.1056822869870071),y |=> 1.5017727799834153
      currMode: @@default@loc@@]]
  dr3 ->[
    (ctrl . ctrlProc . cThread) ->[
      variables: none
      currState: init]
    env ->[
      variables: (velx |=> -5.1264425029806889e+3),(vely |=>
    5.6822869870071963e+3),(x |=> 1.4946276356150925),y |=> 1.1050648024107945
      currMode: @@default@loc@@]]
  dr2 ->[
    (ctrl . ctrlProc . cThread) ->[
      variables: none
      currState: init]
    env ->[
      variables: (velx |=> -5.1264425029806889e+3),(vely |=>
    5.6822869870071963e+3),(x |=> -1.500197794616734),y |=> -1.10528331552988
      currMode: @@default@loc@@]]
  dr4 ->[
    (ctrl . ctrlProc . cThread) ->[
      variables: none
      currState: init]
    env ->[
      variables: (velx |=> -5.1264425029806889e+3),(vely |=>
    5.6822869870071963e+3),(x |=> -1.0951179754513125),y |=>
    -1.4975900021166517
      currMode: @@default@loc@@]]])
```

Problems | Properties | AADL Property Values | HybridSynchAADL Result

PSPC File	Property Id	Result	Method	CPUTim	RunningTime	Location
FourDronesSystem_impl_Instance.pspc	safety	Counterexample found	random	1324ms	1780ms	/FourDronesSyste
FourDronesSystem_impl_Instance.pspc	rendezvous	Reachable	symbolic	1528ms	2250ms	/FourDronesSyste

Fig. 12. HybridSynchAADL Analysis Results

4 Examples

We have developed a variety of HybridSynchAADL models for networked thermostat controllers, and both rendezvous and formation control of drones with respect to single-integrator and double-integrator dynamics. All these models are available at `https://hybridsynchaadl.github.io/benchmark`.

4.1 Networked Thermostat

There are two thermostats that control the temperatures of two rooms located in different places. The goal is to maintain similar temperatures in both rooms. For this purpose, the controllers communicate with each other over a network, and turn the heaters on or off, based on the current temperature of the room and the temperature of the other room. Figure 13 shows the architecture of this networked thermostat system. For room i, for $i = 1, 2$, the controller `ctrl`i controls its environment `env`i (using "connections" explained below).

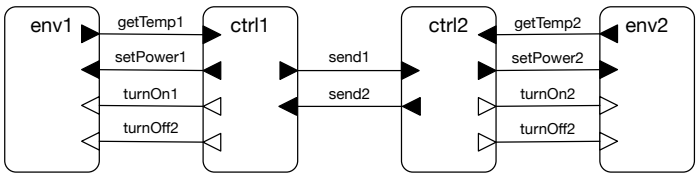

Fig. 13. A networked thermostat system.

The HybridSynchAADL Model. Figure 14 shows an implementation of a top-level system component `TwoThermostats` of our networked thermostat system, depicted in Figure 13. This component has no ports and contains two thermostats and their environments. The controller components `ctrl1` and `ctrl2` are implemented using the system component `Thermostat.impl` in Figure 18, and the environment components `env1` and `env2` are given using `RoomEnv.impl` in Figure 15. Each discrete controller `ctrl`i, for $i = 1, 2$, is connected to its environment component `env`i using four connections `turnOn`i, `turnOff`i, `setPower`i, and `getTemp`i. The controllers `ctrl1` and `ctrl2` are connected with each other using delayed data connections `send1` and `send2`.

Environment. Figure 15 gives an environment component `RoomEnv` for our networked thermostat system. It has data output port `temp`, data input port `power`, and event input ports `on_control` and `off_control`. The implementation of `RoomEnv` has two data subcomponents `x` and `p` to denote the temperature of the room and the heater's power, respectively. They represent the state variables of `RoomEnv` with the specified values.

There are two modes `heaterOn` and `heaterOff` with their respective continuous dynamics, specified by `Hybrid_SynchAADL::ContinuousDynamics`, using continuous functions over time parameter t, where `heaterOff` is the initial mode. Because `p` is a constant, `p`'s dynamics `d/dt(p) = 0` is omitted. The value `x` changes continuously according to the mode and the continuous dynamics.

The value of `x` is sent to the controller through the output port `temp`, declared by the connection `port x -> temp`. When a discrete controller sends an actuation command through input ports `power`, `on_control`, and `off_control`, the mode

```
system TwoThermostats
  properties
    Hybrid_SynchAADL::Synchronous => true;
end TwoThermostats;
system implementation TwoThermostats.impl
  subcomponents
    ctrl1: system Thermostat.impl;        ctrl2: system Thermostat.impl;
    env1: system RoomEnv.impl;            env2: system RoomEnv.impl;
  connections
    turnOn1:   port ctrl1.on_control  -> env1.on_control;
    turnOff1:  port ctrl1.off_control -> env1.off_control;
    setPower1: port ctrl1.set_power   -> env1.power;
    getTemp1:  port env1.temp         -> ctrl1.curr;
    send1:     port ctrl1.tout         > ctrl2.tin;
    turnOn2:   port ctrl2.on_control  -> env2.on_control;
    turnOff2:  port ctrl2.off_control -> env2.off_control;
    setPower2: port ctrl2.set_power   -> env2.power;
    getTemp2:  port env2.temp         -> ctrl2.curr;
    send2:     port ctrl2.tout        -> ctrl1.tin;
  properties
    Period => 10 ms;
    Hybrid_SynchAADL::Max_Clock_Deviation => 1 ms;
    Timing => Delayed applies to send1, send2;
end TwoThermostats.impl;
```

Fig. 14. A top level component with two thermostat controllers.

changes according to the mode transitions, and the value of `p` can be updated by the value of input port `power`, declared by the connection `port x -> temp`.

Controller. Consider again our networked thermostat system. Figure 17 shows a controller system component. The system implementation `Thermostat.impl` includes the process component `thermProcess`. As shown in Figure 16 the thread component `thermThread` is declared as subcomponents in `ThermProcess.impl` The input and ouput port of a wrapper component are connected to the ports of the enclosed subcomponent.

Figure 18 shows a thread component `ThermThread` that turns the heater on or off depending on the average value `avg` of the current temperatures of the two rooms. It has event output ports `on_control` and `off_control`, data input ports `curr` and `tin`, and data output ports `set_power` and `tout`. The ports `on_control`, `off_control`, `set_power`, and `curr` are eventually connected to an environment, and `tin` and `tout` are connected to another controller component (see Fig. 14). The implementation has the data subcomponent `avg` whose initial value is declared as a parameter.

When the thread dispatches, the transition from state `init` to `exec` is taken, which updates `avg` using the values of the input ports `curr` and `tin`, and assigns

```
system RoomEnv
  features
    temp: out data port Base_Types::Float;
    power: in data port Base_Types::Float;
    on_control: in event port;
    off_control: in event port;
  properties
    Hybrid_SynchAADL::isEnvironment => true;
end RoomEnv;

system implementation RoomEnv.impl
  subcomponents
    x: data Base_Types::Float {Data_Model::Initial_Value => ("param");};
    p: data Base_Types::Float {Data_Model::Initial_Value => ("5");};
  connections
    C: port x -> temp;                  R: port power -> p;
  modes
    heaterOff: initial mode;            heaterOn: mode;
    heaterOff -[on_control]-> heaterOn; heaterOn -[off_control]-> heaterOff;
  properties
    Hybrid_SynchAADL::ContinuousDynamics =>
      "x(t) = x(0) - 0.1 * (x(0) - p / 0.1) * t;" in modes (heaterOn),
      "x(t) = x(0) * (1 - 0.1 * t);" in modes (heaterOff);
end RoomEnv.impl;
```

Fig. 15. A RoomEnv component.

to the output port tout the value of curr. Since exec is not a complete state, the thread continues executing by taking one of the other transitions, which may send an event. For example, if the value of avg is smaller than 10, a control command that sets the heater's power to 5 is sent through the port set_power, and an event is sent through the port off_control. The resulting state init is a complete state, and the execution of the current dispatch ends.

Property Specifications. Consider the thermostat system that consists of two thermostat controllers ctrl1 and ctrl2 and their environments env1 and env2, respectively. The following declares two propositions inRan1 and inRan2 using the property specification language. For example, inRan1 holds if the value of env1's data subcomponent x is between 10 and 25.

```
proposition [inRan1]: env1 | (x > 10 and x <= 25);
proposition [inRan2]: env2 | (x > 5  and x <= 10);
```

The following declares the invariant property inv. The initial condition states that the value of env1's data subcomponent x satisfies $|x - 15| < 3$ and the value of env2's data subcomponent x satisfies $|x - 7| < 1$. This property holds if for

```
process ThermProcess
  features
    on_control: out event port;
    off_control: out event port;
    set_power: out data port Base_Types::Float;
    curr: in data port Base_Types::Float;
    tin: in data port Base_Types::Float;
    tout: out data port Base_Types::Float;
end ThermProcess;
process implementation ThermProcess.impl
  subcomponents
    thermThread : thread ThermThread.impl;
  connections
    C1: port thermThread.on_control -> on_control;
    C2: port thermThread.off_control -> off_control;
    C3: port thermThread.set_power -> set_power;
    C4: port thermThread.tout -> tout;
    C5: port curr -> thermThread.curr;
    C6: port tin -> thermThread.tin;
end ThermProcess.impl;
```

Fig. 16. A simple thermostat process.

```
system Thermostat
  features
    on_control: out event port;    off_control: out event port;
    set_power: out data port Base_Types::Float
               {Data_Model::Initial_Value => ("0");};
    curr: in data port Base_Types::Float;
    tin: in data port Base_Types::Float;
    tout: out data port Base_Types::Float
              {Data_Model::Initial_Value => ("0");};
end Thermostat;
system implementation Thermostat.impl
  subcomponents
    thermProcess : process ThermProcess.impl;
  connections
    C1: port thermProcess.on_control -> on_control;
    C2: port thermProcess.off_control -> off_control;
    C3: port thermProcess.set_power -> set_power;
    C4: port thermProcess.tout -> tout;
    C5: port curr -> thermProcess.curr;
    C6: port tin -> thermProcess.tin;
end Thermostat.impl;
```

Fig. 17. A simple thermostat controller.

```
thread ThermThread
  features
    on_control: out event port;
    off_control: out event port;
    set_power: out data port Base_Types::Float;
    curr: in data port Base_Types::Float;
    tin: in data port Base_Types::Float;
    tout: out data port Base_Types::Float;
  properties
    Dispatch_Protocol => Periodic;
    Hybrid_SynchAADL::Sampling_Time => 1ms .. 5ms;
    Hybrid_SynchAADL::Response_Time => 7ms .. 9ms;
end ThermThread;

thread implementation ThermThread.impl
  subcomponents
    avg : data Base_Types::Float {Data_Model::Initial_Value => ("param");};
  annex behavior_specification{**
    states
      init : initial complete state;    exec : state;
    transitions
      init -[on dispatch]-> exec {
        avg := (tin + curr) / 2; tout := curr };
      exec -[avg > 25]-> init {
        off_control! };
      exec -[avg < 20 and avg >= 10]-> init {
        set_power := 5; on_control! };
      exec -[avg < 10]-> init {
        set_power := 10; on_control! };     **};
end ThermThread.impl;
```

Fig. 18. A simple thermostat thread.

each initial state satisfying the initial condition, any reachable state within the time bound 30 satisfies the conditions `inRan1`, `inRan2`, and `env1.x > env2.x`.

```
invariant [inv]: abs(env1.x - 15) < 3 and abs(env2.x - 7) < 1
     ==> ?inRan1 and ?inRan2 and (env1.x > env2.x) in time 30;
```

As shown in Figure 19, there is a counterexample to `inv` up to bound 30. It takes 400ms in the CPU time and 1055ms in the running time to find a counterexample. The result is obtained by the symbolic analysis. Note that the result can also be obatined by the randomized simulation.

4.2 Rendezvous Drones with Single-Integrator

There are four distributed drones with rendezvous controller for single-integrator dynamics. Figure 20 illustrates the AADL architecture of the model. There are

Problems | Properties | AADL Property Values | HybridSynchAADL Result

PSPC File	Property Id	Result	Method	CPUTime	RunningTime	Location
TwoThermostats_impl_Instance.pspc	inv	Counterexample found	symbolic	400ms	1055ms	/TwoTherm

Fig. 19. The analysis results of thermostat system.

four drone components. Each drone is connected with two other drones to exchange positions. For example, Drone 1 sends its position to Drone 2, and receives the position of Drone 4. A drone component consists of an environment and its controller. An environment component specifies the physical model of the drone, including position and velocity. A controller component interacts with the environment according to the sampling and actuating times. All controllers in the model have the same period.

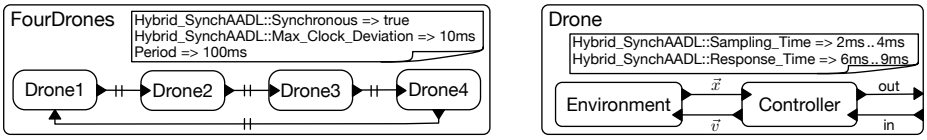

Fig. 20. The AADL architecture of four drones (left), and a drone component (right).

In each round, a controller determines a new velocity to synchronize its movement with the other drones. The controller obtains the position $\vec{x}$ from its environment according to the sampling time. The position of the connected drone is sent in the previous round, and is already available to the controller at the beginning of the round. The controller sends the current position $\vec{x}$ through its output port. In the meantime, the environment changes its position according to the velocity indicated by its controller, where the new velocity $\vec{v}$ from the controller becomes effective according to the actuation time.

The HybridSynchAADL Model. The top-level component includes four `Drone` components (Figure 21). Each drone sends its position through its output ports `oX` and `oY`, and receives the position of the other drone through its input ports `iX` and `iY`. The component is declared to be synchronous with period 100 ms. Also, to meet the constraints of HybridSynchAADL, the connections between drone components are delayed and the output ports have some initial values. The clock skew is given by `Hybrid_SynchAADL::Max_Clock_Deviation`.

Drone Component. A drone component in Figure 22 has input ports `iX` and `iY` and output ports `oX` and `oY`. Its implementation `Drone.impl` contains a controller `ctrl` and an environment `env`. The controller `ctrl` obtains the current position from `env` via input ports `cX` and `cY`, and sends a new velocity to `env` via output ports `vX` and `vY`, according to its sampling and actuating times.

```
system FourDronesSystem
end FourDronesSystem;

system implementation FourDronesSystem.impl
  subcomponents
    dr1: system Drone::Drone.impl;    dr2: system Drone::Drone.impl;
    dr3: system Drone::Drone.impl;    dr4: system Drone::Drone.impl;
  connections
    C1:  port dr1.oX -> dr2.iX;    C2: port dr1.oY -> dr2.iY;
    C3:  port dr2.oX -> dr3.iX;    C4: port dr2.oY -> dr3.iY;
    C5:  port dr3.oX -> dr4.iX;    C6: port dr3.oY -> dr4.iY;
    C7:  port dr4.oX -> dr1.iX;    C8: port dr4.oY -> dr1.iY;
  properties
    Hybrid_SynchAADL::Synchronous => true;
    Period => 100ms;
    Hybrid_SynchAADL::Max_Clock_Deviation => 10ms;
    Timing => Delayed applies to C1, C2, C3, C4, C5, C6, C7, C8;
    Data_Model::Initial_Value => ("0.0") applies to
        dr1.oX, dr2.oX, dr3.oX, dr4.oX,
        dr1.oY, dr2.oY, dr3.oY, dr4.oY;
end FourDronesSystem.impl;
```

Fig. 21. The top-level system component `FourDronesSystem`.

```
system Drone
  features
    iX: in data port Base_Types::Float; oX: out data port Base_Types::Float;
    iY: in data port Base_Types::Float; oY: out data port Base_Types::Float;
end Drone;

system implementation Drone.impl
  subcomponents
    ctrl: system DroneControl::DroneControl.impl;
    env:  system Environment::Environment.impl;
  connections
    C1: port ctrl.oX -> oX;              C2: port ctrl.oY -> oY;
    C3: port iX -> ctrl.iX;              C4: port iY -> ctrl.iY;
    C5: port env.cX -> ctrl.cX;          C6: port env.cY -> ctrl.cY;
    C7: port ctrl.vX -> env.vX;          C8: port ctrl.vY -> env.vY;
  properties
    Hybrid_SynchAADL::Sampling_Time => 2ms .. 4ms;
    Hybrid_SynchAADL::Response_Time => 6ms .. 9ms;
end Drone.impl;
```

Fig. 22. A drone component in HybridSynchAADL.

```
system Environment
  features
    cX: out data port Base_Types::Float;
    cY: out data port Base_Types::Float;
    vX: in data port Base_Types::Float;
    vY: in data port Base_Types::Float;
  properties
    Hybrid_SynchAADL::isEnvironment => true;
end Environment;

system implementation Environment.impl
  subcomponents
    x:  data Base_Types::Float;      y: data Base_Types::Float;
    velx: data Base_Types::Float;    vely: data Base_Types::Float;
  connections
    C1: port x -> cX;           C2: port y -> cY;
    C3: port vX -> velx;        C4: port vY -> vely;
  properties
    Hybrid_SynchAADL::ContinuousDynamics =>
      "x(t) = velx * t + x(0); y(t) = vely * t + y(0);";
    Data_Model::Initial_Value => ("param") applies to x, y, velx, vely;
end Environment.impl;
```

Fig. 23. An environment component in HybridSynchAADL.

Environment. Figure 23 shows an `Environment` component that specifies the physical model of the drone. It has two input ports `vX` and `vY` and two output ports `cX` and `cY`. Data subcomponents `x`, `y`, `velx` and `vely` represent the position and velocity of the drone. The values of `x` and `y` are sent to the controller through the output ports `cX` and `cY`. When a controller sends an actuation command to ports `vX` and `vY`, the values of `velx` and `vely` are updated by the values of `vX` and `vY`, or the mode changes according to the mode transitions. The dynamics of (x, y) is given as continuous functions $x(t) = vel_x t + x(0)$ and $y(t) = vel_y t + y(0)$ over time t in `Hybrid_SynchAADL::ContinuousDynamics`, which are actually equivalent to the ordinary differential equations $\dot{x} = vel_x$ and $\dot{y} = vel_y$.

Controller. Figure 25 shows a controller system component. As explained above, there are four ports `iX`, `iY`, `oX`, and `oY` for communicating with other controllers, and four ports `cX`, `cY`, `vX`, and `vY` for interacting with the environment. The system implementation `DroneControl.impl` includes the process component `ctrlProc`. As shown in Figure 26, `ctrlProc` again includes the thread component `cThread` in its implementation `DroneControlProc.impl`. The input and output ports of a wrapper component (e.g., `ctrlProc`) are connected to the ports of the enclosed subcomponent (e.g., `cThread`).

Figure 27 shows a thread component for a drone controller. When the thread dispatches, the transition from `init` to `exec` is taken. When the distance between

the current position and the connected drone is too close, the new velocity is set to $(0, 0)$ and the `close` flag is set to true to avoid a collision. Otherwise, the new velocity is set toward the connected drone according to a *discretized* version of the distributed consensus algorithm. That is, the new velocity (`vX`, `vY`) is chosen from a predefined set of velocities, according to the value (`nx`, `ny`) obtained by the distributed consensus algorithm and the `close` flag. Finally, the current position is assigned to the output ports `oX` and `oY`.

Property Specifications. Consider two properties of the drone rendezvous model: (i) drones do not collide (`safety`), and (ii) all drones could eventually gather together (`rendezvous`). Because the drone model is a distributed hybrid system, these properties depend on the continuous behavior *perturbed by* sensing and actuating times. We analyze them up to bound 500 ms.

```
invariant [safety]: ?initial and ?velconst ==> not ?collision in time 500;
reachability [rendezvous]: ?initial and ?velconst ==> ?gather in time 500;
```

We define four atomic propositions `collision`, `gather`, `initial`, and `velconst` for four drones `dr1`, `dr2`, `dr3`, and `dr4`. Two drones collide if the distance between them is less than 0.1. All drones have gathered if the distance between each pair of drones is less than 1. The initial values of `x`, `y`, `velx` and `vely` are declared to be parametric in Figure 23 and constrained by the propositions `initial` and `velconst`. There are infinitely many states satisfying the initial condition.

```
proposition [collision]:
   (abs(dr1.env.x - dr2.env.x) < 0.1 and abs(dr1.env.y - dr2.env.y) < 0.1) or
   (abs(dr1.env.x - dr3.env.x) < 0.1 and abs(dr1.env.y - dr3.env.y) < 0.1) or
   (abs(dr1.env.x - dr4.env.x) < 0.1 and abs(dr1.env.y - dr4.env.y) < 0.1) or
   (abs(dr2.env.x - dr3.env.x) < 0.1 and abs(dr2.env.y - dr3.env.y) < 0.1) or
   (abs(dr2.env.x - dr4.env.x) < 0.1 and abs(dr2.env.y - dr4.env.y) < 0.1) or
   (abs(dr3.env.x - dr4.env.x) < 0.1 and abs(dr3.env.y - dr4.env.y) < 0.1);
proposition [gather]:
   abs(dr1.env.x - dr2.env.x) < 1 and abs(dr1.env.y - dr2.env.y) < 1 and
   abs(dr1.env.x - dr3.env.x) < 1 and abs(dr1.env.y - dr3.env.y) < 1 and
   abs(dr1.env.x - dr4.env.x) < 1 and abs(dr1.env.y - dr4.env.y) < 1 and
   abs(dr2.env.x - dr3.env.x) < 1 and abs(dr2.env.y - dr3.env.y) < 1 and
   abs(dr2.env.x - dr4.env.x) < 1 and abs(dr2.env.y - dr4.env.y) < 1 and
   abs(dr3.env.x - dr4.env.x) < 1 and abs(dr3.env.y - dr4.env.y) < 1;
proposition [initial]:
   abs(dr1.env.x - 1.1) < 0.01 and abs(dr1.env.y - 1.5) < 0.01 and
   abs(dr2.env.x + 1.5) < 0.01 and abs(dr2.env.y + 1.1) < 0.01 and
   abs(dr3.env.x - 1.5) < 0.01 and abs(dr3.env.y - 1.1) < 0.01 and
   abs(dr4.env.x + 1.1) < 0.01 and abs(dr4.env.y + 1.5) < 0.01;
proposition [velconst]:
   abs(dr1.env.velx) <= 0.01 and abs(dr1.env.vely) <= 0.01 and
   abs(dr2.env.velx) <= 0.01 and abs(dr2.env.vely) <= 0.01 and
   abs(dr3.env.velx) <= 0.01 and abs(dr3.env.vely) <= 0.01 and
   abs(dr4.env.velx) <= 0.01 and abs(dr4.env.vely) <= 0.01;
```

As shown in Figure 24, there is no counterexample to `safety` up to bound 500. The result is obtained by the symbolic analysis for `safety`, and by the randomized simulation for `rendezvous`.

Problems | Properties | AADL Property Values | HybridSynchAADL Result

PSPC File	Property Id	Result	Method	CPUTime	RunningTime	Location
FourDronesSystem_impl_Instance2.pspc	rendezvous	Reachable	random	92ms	512ms	/FourDrone
FourDronesSystem_impl_Instance2.pspc	safety	No counterexample found	symbolic	642068ms	642898ms	/FourDrone

Fig. 24. The analysis results of drone rendezvous models

4.3 Formation Drones with Double-Integrator

There are four distributed drones with a formation controller for double-integrator dynamics. Compared to the rendezvous drones in Section 4.2, each drone sends its position and velocity to the connected drone. In each round, a controller determines a new acceleration. The controller obtains the position and velocity from its environment. The environment changes its position and velocity according to the acceleration indicated by its controller.

In the case of formation drones, all drones follow a reference drone which changes its acceleration in a predefined set of accelerations in each round. All drones try to keep formation based on the position of the reference drone and offset values.

The HybridSynchAADL Model. The top-level component includes four `Drone` components and one `RefDrone` component as illustrated in Figure 28. Each drone sends its position and velocity through its output ports `oX`, `oY`, `oVX`, and `oVY`, and receives the position and velocity of the other drone through its input ports `iX`, `iY`, `iVX`, and `iVY`. Each drone also receives the position and velocity of the reference drone through its input ports `rX`, `rY`, `rVX`, and `rVY`. To keep the formation, each drone has initialized offset values `offsetX` and `offsetY` in its thread component.

Drone Component. A drone component in Figure 29 has input ports `iX`, `iY`, `iVX`, `iVY`, `rX` and `rY` and output ports `oX`, `oY`, `oVX`, and `oVY`. The controller `ctrl` obtains the current position and velocity from `env` via input ports `cX`, `cY`, `cVX` and `cVY`, and sends a new acceleration to `env` via output ports `aX` and `aY`. The controller `ctrl` obtains the reference drone's position and velocity through its input ports `rX`, `rY`, `rVX`, and `rVY` to calculate a proper acceleration.

Environment. Figure 30 shows an `Environment` component. It has two input ports `aX` and `aY` and output ports `cX`, `cY`, `cVX`, and `cVY`. Data subcomponents `x`, `y`, `velx`, `vely`, `accx`, and `accy` represent the position, velocity and acceleration

```
system DroneControl
  features
    iX: in data port Base_Types::Float;
    iY: in data port Base_Types::Float;
    oX: out data port Base_Types::Float;
    oY: out data port Base_Types::Float;
    cX: in data port Base_Types::Float;
    cY: in data port Base_Types::Float;
    vX: out data port Base_Types::Float;
    vY: out data port Base_Types::Float;
end DroneControl;

system implementation DroneControl.impl
  subcomponents
    ctrlProc: process DroneControlProc.impl;
  connections
    C1: port ctrlProc.oX -> oX;         C2: port ctrlProc.oY -> oY;
    C3: port iX -> ctrlProc.iX;         C4: port iY -> ctrlProc.iY;
    C5: port cX -> ctrlProc.cX;         C6: port cY -> ctrlProc.cY;
    C7: port ctrlProc.vX -> vX;         C8: port ctrlProc.vY -> vY;
end DroneControl.impl;
```

Fig. 25. A controller system component.

```
process DroneControlProc
  features
    iX: in data port Base_Types::Float;
    iY: in data port Base_Types::Float;
    oX: out data port Base_Types::Float;
    oY: out data port Base_Types::Float;
    cX: in data port Base_Types::Float;
    cY: in data port Base_Types::Float;
    vX: out data port Base_Types::Float;
    vY: out data port Base_Types::Float;
end DroneControlProc;

process implementation DroneControlProc.impl
  subcomponents
    cThread: process DroneControlThread.impl;
  connections
    C1: port cThread.oX -> oX;         C2: port cThread.oY -> oY;
    C3: port iX -> cThread.iX;         C4: port iY -> cThread.iY;
    C5: port cX -> cThread.cX;         C6: port cY -> cThread.cY;
    C7: port cThread.vX -> vX;         C8: port cThread.velY -> vY;
end DroneControlProc.impl;
```

Fig. 26. A controller process component

```
thread DroneControlThread
  features
    iX: in data port Base_Types::Float;
    iY: in data port Base_Types::Float;
    oX: out data port Base_Types::Float;
    oY: out data port Base_Types::Float;
    cX: in data port Base_Types::Float;
    cY: in data port Base_Types::Float;
    vX: out data port Base_Types::Float;
    vY: out data port Base_Types::Float;
  properties
    Dispatch_Protocol => Periodic;
end DroneControlThread;

thread implementation DroneControlThread.impl
  subcomponents
    close: data Base_Types::Boolean
                          {Data_Model::Initial_Value => ("false");};
  annex behavior_specification {**
    variables
      nx, ny : Base_Types::Float;
    states
      init: initial complete state; exec, output: state;
    transitions
      init -[on dispatch]-> exec;
      exec -[abs(cX - iX) < 0.5 and abs(cY - iY) < 0.5]-> output {
        vX := 0; vY := 0; close := true
      };
      exec -[otherwise]-> output {
        nx := -#DroneSpec::A * (cX - iX);
        ny := -#DroneSpec::A * (cY - iY);
        if (nx > 0.3)      vX := 2.5
        elsif (nx > 0.15)
          if (close)       vX := 1.5
          else             vX := 0.0
          end if
        else               vX := -2.5
        end if;
        if (ny > 0.3)      vY := 2.5
        elsif (ny > 0.15)
          if (close)       vY := 1.5
          else             vY := 0.0
          end if
        else               vY := -2.5
        end if;
        close := false };
      output -[ ]-> init { oX := cX; oY := cY };
  **};
end DroneControlThread.impl;
```

Fig. 27. A controller thread in HybridSynchAADL

```
system FourDronesSystem
end FourDronesSystem;

system implementation FourDronesSystem.impl
  subcomponents
    dr1: system Drone::Drone.impl;  dr3: system Drone::Drone.impl;
    dr2: system Drone::Drone.impl;  dr4: system Drone::Drone.impl;
    refDr: system RefDrone::RefDrone.impl;
  connections
    d1x: port dr1.oX -> dr2.iX;       d1y: port dr1.oY -> dr2.iY;
    d2x: port dr2.oX -> dr3.iX;       d2y: port dr2.oY -> dr3.iY;
    d3x: port dr3.oX -> dr4.iX;       d3y: port dr3.oY -> dr4.iY;
    d4x: port dr4.oX -> dr1.iX;       d4y: port dr4.oY -> dr1.iY;
    d1vx: port dr1.oVX -> dr2.iVX;    d1vy: port dr1.oVY -> dr2.iVY;
    d2vx: port dr2.oVX -> dr3.iVX;    d2vy: port dr2.oVY -> dr3.iVY;
    d3vx: port dr3.oVX -> dr4.iVX;    d3vy: port dr3.oVY -> dr4.iVY;
    d4vx: port dr4.oVX -> dr1.iVX;    d4vy: port dr4.oVY -> dr1.iVY;
    r1x: port refDr.oX -> dr1.rX;     r1y: port refDr.oY -> dr1.rY;
    r2x: port refDr.oX -> dr2.rX;     r2y: port refDr.oY -> dr2.rY;
    r3x: port refDr.oX -> dr3.rX;     r3y: port refDr.oY -> dr3.rY;
    r4x: port refDr.oX -> dr4.rX;     r4y: port refDr.oY -> dr4.rY;
    r1vx: port refDr.oVX -> dr1.rVX;  r1vy: port refDr.oVY -> dr1.rVY;
    r2vx: port refDr.oVX -> dr2.rVX;  r2vy: port refDr.oVY -> dr2.rVY;
    r3vx: port refDr.oVX -> dr3.rVX;  r3vy: port refDr.oVY -> dr3.rVY;
    r4vx: port refDr.oVX -> dr4.rVX;  r4vy: port refDr.oVY -> dr4.rVY;

  properties
    Data_Model::Initial_Value => ("-0.5") applies to
      dr1.drone.droneProc.droneThread.offsetX,
      dr1.drone.droneProc.droneThread.offsetY,
      dr2.drone.droneProc.droneThread.offsetY;
    Data_Model::Initial_Value => ("0") applies to
      dr2.drone.droneProc.droneThread.offsetX,
      dr4.drone.droneProc.droneThread.offsetX,
      dr4.drone.droneProc.droneThread.offsetY;
    Data_Model::Initial_Value => ("0.5") applies to
      dr3.drone.droneProc.droneThread.offsetX,
      dr3.drone.droneProc.droneThread.offsetY;

    Hybrid_SynchAADL::Max_Clock_Deviation => 5ms;
    Hybrid_SynchAADL::Synchronous => true;
    Period => 100ms;
    Timing => Delayed applies to
      d1x,  d2x, d3x, d4x, d1vx, d2vx, d3vx, d4vx,
      r1x,  r2x, r3x, r4x, r1vx, r2vx, r3vx, r4vx,
      d1y,  d2y, d3y, d4y, d1vy, d2vy, d3vy, d4vy,
      r1y,  r2y, r3y, r4y, r1vy, r2vy, r3vy, r4vy;
end FourDronesSystem.impl;
```

Fig. 28. The top-level system component `FourDronesSystem`.

```
system Drone
  features
   iX: in data port Base_Types::Float;  iY: in data port Base_Types::Float;
   iVX:in data port Base_Types::Float; iVY: in data port Base_Types::Float;
   rX: in data port Base_Types::Float;  rY: in data port Base_Types::Float;
   rVX:in data port Base_Types::Float; rVY: in data port Base_Types::Float;
   oX: out data port Base_Types::Float {Data_Model::Initial_Value => ("0");};
   oVX:out data port Base_Types::Float {Data_Model::Initial_Value => ("0");};
   oY: out data port Base_Types::Float {Data_Model::Initial_Value => ("0");};
   oVY:out data port Base_Types::Float {Data_Model::Initial_Value => ("0");};
end Drone;

system implementation Drone.impl
  subcomponents
    ctrl: system DroneControl::DroneControl.impl;
    env: system Environment::Environment.impl;
  connections
    C1: port ctrl.oX -> oX;        C10: port ctrl.oY -> oY;
    C2: port ctrl.oVX -> oVX;      C11: port ctrl.oVY -> oVY;
    C3: port iX -> ctrl.iX;        C12: port iY -> ctrl.iY;
    C4: port iVX -> ctrl.iVX;      C13: port iVY -> ctrl.iVY;
    C5: port ctrl.aX -> env.accX;  C14: port ctrl.aY -> env.accY;
    C6: port env.cX -> ctrl.cX;    C15: port env.cY -> ctrl.cY;
    C7: port env.cVX -> ctrl.cVX;  C16: port env.cVY -> ctrl.cVY;
    C8: port rX -> ctrl.rX;        C17: port rY -> ctrl.rY;
    C9: port rVX -> ctrl.rVX;      C18: port rVY -> ctrl.rVY;
  properties
    Hybrid_SynchAADL::Sampling_Time => 3 ms .. 5 ms;
    Hybrid_SynchAADL::Response_Time => 20 ms .. 30 ms;
end Drone.impl;
```

Fig. 29. A drone component in HybridSynchAADL.

of the drone. The dynamics of (x, y) is given as continuous functions $x(t) = x(0) + vel_x t + 1/2\ acc_x t^2$ and $y(t) = y(0) + vel_y t + 1/2\ acc_y t^2$ over time t in `Hybrid_SynchAADL::ContinuousDynamics`. The dynamics of (vel_x, vel_y) is also given as $vel_x(t) = vel_x(0) + acc_x t$ and $vel_y(t) = vel_y(0) + acc_y t$.

```
system Environment
  features
    cX: out data port Base_Types::Float;  cY: out data port Base_Types::Float;
    cVX: out data port Base_Types::Float; cVY: out data port Base_Types::Float;
    aX: in data port Base_Types::Float;   aY: in data port Base_Types::Float;
  properties
    Hybrid_SynchAADL::isEnvironment => true;
end Environment;

system implementation Environment.impl
  subcomponents
    x : data Base_Types::Float {Data_Model::Initial_Value => ("param");};
    y : data Base_Types::Float {Data_Model::Initial_Value => ("param");};
    velx : data Base_Types::Float {Data_Model::Initial_Value => ("0");};
    vely : data Base_Types::Float {Data_Model::Initial_Value => ("0");};
    accx : data Base_Types::Float {Data_Model::Initial_Value => ("0");};
    accy : data Base_Types::Float {Data_Model::Initial_Value => ("0");};
  connections
    C1: port velx -> cVX; C4: port vely -> cVY;
    C2: port x -> cX;     C5: port y -> cY;
    C3: port aX -> accx;  C6: port aY -> accy;
  properties
   Hybrid_SynchAADL::ContinuousDynamics =>
   "velx(t) = ((0.001) * accx * t) + velx(0);
    x(t) = (x(0) + (0.001 * velx(0) * t) + ((0.000001) * accx * t * t) / 2);
    vely(t) = ((0.001) * accy * t) + vely(0);
    y(t) = (y(0) + (0.001 * vely(0) * t) + ((0.000001) * accy * t * t) / 2);";
end Environment.impl;
```

Fig. 30. An environment component in HybridSynchAADL.

Drone Controller. Figure 31 shows a thread component for a drone controller. When the distance between the current position and the connected drone is too close, the new acceleration is set to negation of the current velocity. Otherwise, the new acceleration is set toward the connected drone according to a *discretized* version of the distributed consensus algorithm. At the `output` state, the thread component saves the current velocity of the reference drone.

Reference Drone Controller. Figure 32 shows a thread component for a reference drone. It saves its states using data subcomponents `nx` and `ny` which represents

```
thread DroneControlThread
  features
   cX: in data port Base_Types::Float;   cY: in data port Base_Types::Float;
   cVX:in data port Base_Types::Float;  cVY: in data port Base_Types::Float;
   iX: in data port Base_Types::Float;   iY: in data port Base_Types::Float;
   iVX:in data port Base_Types::Float;  iVY: in data port Base_Types::Float;
   oX :out data port Base_Types::Float; oY : out data port Base_Types::Float;
   oVX:out data port Base_Types::Float; oVY: out data port Base_Types::Float;
   aX: out data port Base_Types::Float;  aY: out data port Base_Types::Float;
   rX: in data port Base_Types::Float;   rY: in data port Base_Types::Float;
   rVX:in data port Base_Types::Float;  rVY: in data port Base_Types::Float;
  properties Dispatch_Protocol => Periodic;
end DroneControlThread;
thread implementation DroneControlThread.impl
  subcomponents
    offsetX: data Base_Types::Float;
    offsetY: data Base_Types::Float;
    rVX0: data Base_Types::Float {Data_Model::Initial_Value => ("0");};
    rVY0: data Base_Types::Float {Data_Model::Initial_Value => ("0");};
  annex behavior_specification {**
    variables
      nx, ny, raX, raY : Base_Types::Float;
    states
      init : initial complete state;
      exec, output : state;
    transitions
    init -[on dispatch]-> exec;
    exec -[abs(cX - iX) < 0.3 and abs(cY - iY) < 0.3]-> output{
      aX := -cVX; aY := -cVY };
    exec -[otherwise]-> output {
      raX := (rVX - rVX0);
      nx := raX - #DroneSpec::alpha *
       (cX - offsetX - rX + #DroneSpec::gamma * (cVX - rVX)) -
     - #DroneSpec::A * (cX - offsetX - iX + #DroneSpec::gamma * (cVX - iVX));
      raY := (rVY - rVY0);
      ny := raY - #DroneSpec::alpha *
       (cY- offsetY - rY + #DroneSpec::gamma * (cVY - rVY))
     - #DroneSpec::A * (cY - offsetY - iY + #DroneSpec::gamma * (cVY - iVY));
      if (nx > 0.5)    aX := 40
      elsif (nx > 0)   aX := 0
      else             aX := -40 end if;
      if (ny > 0.5)    aY := 40
      elsif (ny > 0)   aY := 0
      else             aY := -40 end if };
    output -[ ]-> init {
        oX := cX - offsetX;  oY := cY - offsetY;
        oVX := cVX;          oVY := cVY;
        rVX0 := rVX;         rVY0 := rVY };
**};
end DroneControlThread.impl;
```

Fig. 31. A controller thread in HybridSynchAADL

```
thread RefDroneThread
 features
  aX:  out data port Base_Types::Float; aY:  out data port Base_Types::Float;
  oX:  out data port Base_Types::Float; oY:  out data port Base_Types::Float;
  oVX: out data port Base_Types::Float; oVY: out data port Base_Types::Float;
  cX:  in data port Base_Types::Float;  cY:  in data port Base_Types::Float;
  cVX: in data port Base_Types::Float;  cVY: in data port Base_Types::Float;
 properties
      Dispatch_Protocol => Periodic;
end RefDroneThread;
thread implementation RefDroneThread.impl
  subcomponents
    nx : data Base_Types::Float {Data_Model::Initial_Value => ("0");};
    ny : data Base_Types::Float {Data_Model::Initial_Value => ("0");};
  annex behavior_specification {**
    states
      init : initial complete state;
      exec, output : state;
    transitions
      init -[ on dispatch ]-> exec;
      exec -[ nx = 0 and ny = 0]-> output {
        nx := 1; ny := 1 };
      exec -[ nx = 1 and ny = 1]-> output {
        nx := 2; ny := 2 };
      exec -[ nx = 2 and ny = 2]-> output {
        nx := 0; ny := 0 };
      output -[ ]-> init {
        aX := nx;    aY := ny;
        oX := cX;    oY := cY;
        oVX := cVX;  oVY := cVY
      };
  **};
end RefDroneThread.impl;
```

Fig. 32. A reference drone controller in HybridSynchAADL

the current acceleration of the reference drone. Eventually, the acceleration of the reference drone is changed into one of $\{0, 1, 2\}$ in each period.

Property Specifications. In the formation drone model, consider only one property: "drones do not collide". We analyze them up to bound 500 ms. Compared to the drone rendezvous model, we add the initial constraints of the reference drone in `initial`. Similar to the drone rendezvous model, there are many initial states satisfying the proposition `initial`.

```
invariant [safety]: ?initial ==> not ?collision in time 500;

proposition [collision]:
   (abs(dr1.env.x - dr2.env.x) < 0.1 and abs(dr1.env.y - dr2.env.y) < 0.1) or
   (abs(dr1.env.x - dr3.env.x) < 0.1 and abs(dr1.env.y - dr3.env.y) < 0.1) or
   (abs(dr1.env.x - dr4.env.x) < 0.1 and abs(dr1.env.y - dr4.env.y) < 0.1) or
   (abs(dr2.env.x - dr3.env.x) < 0.1 and abs(dr2.env.y - dr3.env.y) < 0.1) or
   (abs(dr2.env.x - dr4.env.x) < 0.1 and abs(dr2.env.y - dr4.env.y) < 0.1) or
   (abs(dr3.env.x - dr4.env.x) < 0.1 and abs(dr3.env.y - dr4.env.y) < 0.1);

proposition [initial]:
   abs(dr1.env.x - 1.1) < 0.01 and abs(dr1.env.y - 1.5) < 0.01 and
   abs(dr2.env.x + 1.5) < 0.01 and abs(dr2.env.y + 1.1) < 0.01 and
   abs(dr3.env.x - 1.5) < 0.01 and abs(dr3.env.y - 1.1) < 0.01 and
   abs(dr4.env.x + 1.1) < 0.01 and abs(dr4.env.y + 1.5) < 0.01 and
   abs(refDr.env.x - 0.0) < 0.01 and abs(refDr.env.y - 0.0) < 0.01;
```

As shown in Figure 33, there is a counterexample to `safety`. The result is obtained by the randomized simulation method. The elapssed CPU time and running time to get the result are 216ms and 782ms respectively.

Problems | Properties | AADL Property Values | HybridSynchAADL Result

PSPC File	Property Id	Result	Method	CPUTime	RunningTime	Location
FourDronesSystem_impl_Instance.pspc	safety	Counterexample found	random	216ms	782ms	/FourDron

Fig. 33. The analysis results of drone formation models

References

1. Supplementary material: HybridSynchAADL technical report, semantics, benchmarks, and the tool, `https://hybridsynchaadl.github.io/`
2. Abrial, J., Börger, E., Langmaack, H. (eds.): Formal Methods for Industrial Applications: Specifying and Programming the Steam Boiler Control, LNCS, vol. 1165. Springer (1996)

3. Ahmad, E., Dong, Y., Wang, S., Zhan, N., Zou, L.: Adding formal meanings to AADL with Hybrid Annex. In: Proc. FACS. LNCS, vol. 8997. Springer (2015)
4. Ahmad, E., Larson, B.R., Barrett, S.C., Zhan, N., Dong, Y.: Hybrid Annex: an AADL extension for continuous behavior and cyber-physical interaction modeling. In: Proc. ACM SIGAda annual conference on High integrity language technology (HILT'14). ACM (2014)
5. Al-Nayeem, A., Sun, M., Qiu, X., Sha, L., Miller, S.P., Cofer, D.D.: A formal architecture pattern for real-time distributed systems. In: Proc. RTSS. IEEE (2009)
6. Kong, S., Gao, S., Chen, W., Clarke, E.M.: dReach: δ-reachability analysis for hybrid systems. In: Proc. TACAS. Lecture Notes in Computer Science, vol. 7898. Springer (2015)
7. Bae, K., Gao, S.: Modular SMT-based analysis of nonlinear hybrid systems. In: Proc. FMCAD. pp. 180–187. IEEE (2017)
8. Bae, K., Meseguer, J., Ölveczky, P.C.: Formal patterns for multirate distributed real-time systems. Science of Computer Programming **91**, 3–44 (2014)
9. Bae, K., Ölveczky, P.C., Al-Nayeem, A., Meseguer, J.: Synchronous aadl and its formal analysis in real-time maude. In: International Conference on Formal Engineering Methods. pp. 651–667. Springer (2011)
10. Bae, K., Ölveczky, P.C., Al-Nayeem, A., Meseguer, J.: Synchronous AADL and its formal analysis in Real-Time Maude. In: Proc. ICFEM'11. LNCS, vol. 6991. Springer (2011)
11. Bae, K., Ölveczky, P.C., Kong, S., Gao, S., Clarke, E.M.: SMT-based analysis of virtually synchronous distributed hybrid systems. In: Proc. HSCC. ACM (2016)
12. Bae, K., Ölveczky, P.C., Meseguer, J.: Definition, semantics, and analysis of multirate synchronous aadl. In: Proc. FM. Lecture Notes in Computer Science, vol. 8442, pp. 94–109. Springer (2014)
13. Bae, K., Ölveczky, P.C., Meseguer, J.: Definition, semantics, and analysis of Multirate Synchronous AADL. In: Proc. FM'14. LNCS, vol. 8442. Springer (2014)
14. Bae, K., Ölveczky, P.C., Meseguer, J., Al-Nayeem, A.: The SynchAADL2Maude tool. In: Proc. FASE'12. LNCS, vol. 7212. Springer (2012)
15. Bae, K., Rocha, C.: Symbolic state space reduction with guarded terms for rewriting modulo SMT. Science of Computer Programming **178**, 20–42 (2019)
16. Bak, S., Bogomolov, S., Johnson, T.T.: Hyst: a source transformation and translation tool for hybrid automaton models. In: Proceedings of the 18th International Conference on Hybrid Systems: Computation and Control. pp. 128–133 (2015)
17. Bak, S., Duggirala, P.S.: Hylaa: A tool for computing simulation-equivalent reachability for linear systems. In: Proc. HSCC. pp. 173–178 (2017)
18. Bak, S., Tran, H.D., Johnson, T.T.: Numerical verification of affine systems with up to a billion dimensions. In: Proceedings of the 22nd ACM International Conference on Hybrid Systems: Computation and Control. pp. 23–32 (2019)
19. Bao, Y., Chen, M., Zhu, Q., Wei, T., Mallet, F., Zhou, T.: Quantitative performance evaluation of uncertainty-aware Hybrid AADL designs using statistical model checking. IEEE Transactions on CAD of Integrated Circuits and Systems **36**(12), 1989–2002 (2017)
20. Barrett, C., Conway, C.L., Deters, M., Hadarean, L., Jovanović, D., King, T., Reynolds, A., Tinelli, C.: CVC4. In: CAV. pp. 171–177. Springer (2011)
21. Baudart, G., Bourke, T., Pouzet, M.: Soundness of the quasi-synchronous abstraction. In: Proc. FMCAD. pp. 9–16. IEEE (2016)
22. Caspi, P., Mazuet, C., Paligot, N.R.: About the design of distributed control systems: The quasi-synchronous approach. In: International Conference on Computer Safety, Reliability, and Security. Springer (2001)

23. Chen, X., Ábrahám, E., Sankaranarayanan, S.: Flow*: An analyzer for non-linear hybrid systems. In: Proc. CAV. pp. 258–263. Springer (2013)
24. Cimatti, A., Griggio, A., Mover, S., Tonetta, S.: HyComp: An SMT-based model checker for hybrid systems. In: Proc. TACAS. LNCS, vol. 9035. Springer (2015)
25. Clavel, M., Durán, F., Eker, S., Meseguer, J., Lincoln, P., Martí-Oliet, N., Talcott, C.: All About Maude – A High-Performance Logical Framework, Lecture Notes in Computer Science, vol. 4350. Springer (2007)
26. Desai, A., Seshia, S.A., Qadeer, S., Broman, D., Eidson, J.C.: Approximate synchrony: An abstraction for distributed almost-synchronous systems. In: Proc. CAV'15. LNCS, vol. 9207. Springer (2015)
27. Dutertre, B.: Yices 2.2. In: Biere, A., Bloem, R. (eds.) CAV. LNCS, vol. 8559, pp. 737–744. Springer (July 2014)
28. Feiler, P.H., Gluch, D.P.: Model-Based Engineering with AADL: An Introduction to the SAE Architecture Analysis and Design Language. Addison-Wesley (2012)
29. França, R., Bodeveix, J.P., Filali, M., Rolland, J.F., Chemouil, D., Thomas, D.: The AADL Behaviour Annex - experiments and roadmap. In: Proc. ICECCS'07. IEEE (2007)
30. Frehse, G., Guernic, C.L., Donzé, A., Cotton, S., Ray, R., Lebeltel, O., Ripado, R., Girard, A., Dang, T., Maler, O.: SpaceEx: Scalable verification of hybrid systems. In: Proc. CAV. LNCS, vol. 6806. Springer (2011)
31. Gao, S., Kong, S., Clarke, E.M.: dReal: An SMT solver for nonlinear theories over the reals. In: Proc. CADE. Lecture Notes in Computer Science, vol. 7898. Springer (2013)
32. Girault, A., Ménier, C.: Automatic production of globally asynchronous locally synchronous systems. In: International Workshop on Embedded Software. pp. 266–281. Springer (2002)
33. Halbwachs, N., Mandel, L.: Simulation and verification of asynchronous systems by means of a synchronous model. In: Sixth International Conference on Application of Concurrency to System Design (ACSD'06). pp. 3–14. IEEE (2006)
34. Henzinger, T.: The theory of hybrid automata. In: Verification of Digital and Hybrid Systems, NATO ASI Series, vol. 170, pp. 265–292. Springer (2000)
35. Kuznetsov, V., Kinder, J., Bucur, S., Candea, G.: Efficient state merging in symbolic execution. Acm Sigplan Notices **47**(6), 193–204 (2012)
36. Larrieu, R., Shankar, N.: A framework for high-assurance quasi-synchronous systems. In: 2014 Twelfth ACM/IEEE Conference on Formal Methods and Models for Codesign (MEMOCODE). pp. 72–83. IEEE (2014)
37. Liu, J., Li, T., Ding, Z., Qian, Y., Sun, H., He, J.: AADL+: a simulation-based methodology for cyber-physical systems. Frontiers Comput. Sci. **13**(3), 516–538 (2019)
38. Meseguer, J.: Conditional rewriting logic as a unified model of concurrency. Theoretical Computer Science **96**(1), 73–155 (1992)
39. Meseguer, J., Ölveczky, P.C.: Formalization and correctness of the PALS architectural pattern for distributed real-time systems. Theoretical Computer Science **451**, 1–37 (2012)
40. Miller, S., Cofer, D., Sha, L., Meseguer, J., Al-Nayeem, A.: Implementing logical synchrony in integrated modular avionics. In: Proc. IEEE/AIAA 28th Digital Avionics Systems Conference. IEEE (2009)
41. Ölveczky, P.C., Boronat, A., Meseguer, J.: Formal semantics and analysis of behavioral aadl models in real-time maude. In: Formal Techniques for Distributed Systems, pp. 47–62. Springer (2010)

42. Potop-Butucaru, D., Caillaud, B.: Correct-by-construction asynchronous implementation of modular synchronous specifications. Fundamenta Informaticae **78**(1), 131–159 (2007)
43. Qian, Y., Liu, J., Chen, X.: Hybrid AADL: a sublanguage extension to AADL. In: Proc. Internetware'13. ACM (2013)
44. Raisch, J., Klein, E., Meder, C., Itigin, A., O'Young, S.: Approximating automata and discrete control for continuous systems — two examples from process control. In: Hybrid systems V. pp. 279–303. Springer (1999)
45. Ren, W., Beard, R.W.: Distributed consensus in multi-vehicle cooperative control. Springer (2008)
46. Rocha, C., Meseguer, J., Muñoz, C.: Rewriting modulo SMT and open system analysis. Journal of Logical and Algebraic Methods in Programming **86**(1), 269–297 (2017)
47. Rushby, J.: Systematic formal verification for fault-tolerant time-triggered algorithms. IEEE Transactions on Software Engineering **25**(5), 651–660 (1999)
48. Tripakis, S., Pinello, C., Benveniste, A., Sangiovanni-Vincent, A., Caspi, P., Di Natale, M.: Implementing synchronous models on loosely time triggered architectures. IEEE Transactions on Computers **57**(10), 1300–1314 (2008)